Systematic Analysis of Browser Extension Malware Detection Methodologies in Top-Tier Security Conferences

Introduction to the Browser Extension Threat Landscape

The modern web browser has evolved from a simple document viewer into a highly complex, quasi-operating system. This transformation is primarily driven by the extensibility afforded by browser extensions, which are software packages written in standard web technologies—HTML, Cascading Style Sheets (CSS), and JavaScript—governed by a centralized manifest file. While these extensions greatly enhance user productivity by enabling ad blocking, password management, and generative artificial intelligence (GenAI) integrations, they simultaneously introduce a vast and highly privileged attack surface. Browser extensions operate within a unique security architecture that grants them deep access to the Document Object Model (DOM), the ability to intercept and modify raw network traffic, and permissions to invoke underlying browser Application Programming Interfaces (APIs). Because of these elevated privileges, malicious extensions have become one of the most prevalent methods used to compromise internet security, allowing threat actors to hijack session cookies, arbitrarily change the content of websites, steal user data, and expose users to malvertising[1].

The primary distribution platform for these software packages, the Chrome Web Store (CWS), has historically struggled to vet submissions effectively. Despite Google's implementation of automated vetting processes, significant gaps remain. Research indicates that a large percentage of infringing extensions are extremely similar to previously vetted items, and these malicious extensions often remain active in the store for months or even years before being removed[1]. The transition from Manifest V2 to Manifest V3 was intended to mitigate some of these risks by restricting powerful APIs—such as replacing the synchronous webRequest blocking API with declarativeNetRequest and forbidding the execution of remotely hosted code[6]. However, sophisticated attackers continuously adapt, finding new avenues for abuse even within the constrained Manifest V3 environment[6].

Detecting malicious browser extensions is a fundamentally difficult machine learning and program analysis challenge. The dynamic nature of JavaScript allows malicious logic to be obfuscated, dynamically evaluated, or fetched at runtime. Furthermore, extensions employ a complex, multi-process architecture comprising background service workers, content scripts injected into web pages, and user interface popups. Attackers leverage these complexities to employ sophisticated evasion techniques, such as delayed activation, dilatory tactics, and bait-and-switch updates, which easily bypass traditional signature-based antivirus engines[2].

The academic security community—particularly in top-tier venues such as the IEEE Symposium on Security and Privacy (S&P), the USENIX Security Symposium, the Network and Distributed System Security Symposium (NDSS), and the ACM Conference on Computer and Communications Security (CCS)—has dedicated significant effort to characterizing and detecting these threats. This report provides an exhaustive, narrative-driven survey of seminal and state-of-the-art security papers in this domain. By deconstructing their threat models, static and dynamic feature engineering pipelines, dataset curation strategies, evaluation protocols, and machine learning model selections, this analysis synthesizes the foundational methodologies required to build robust, academically rigorous malware detectors.

Deconstructing Threat Models in Top-Tier Research

Before proposing a detection architecture, rigorous security research mandates the explicit definition of a threat model. The threat model dictates the assumed capabilities of the adversary, the attack vector, and the ultimate objective of the malicious behavior. In the realm of browser extensions, academic literature generally categorizes threats into several distinct, highly formalized paradigms.

The Bait-and-Switch and Update Delta Model

A prominent and highly effective vector for extension malware is the "bait-and-switch" attack, heavily analyzed in the ACM CCS 2020 paper *You've Changed: Detecting Malicious Browser Extensions through their Update Deltas*[1]. In this threat model, the adversary's primary obstacle is the initial vetting process of the official extension marketplace. To circumvent this, the attacker initially publishes a completely benign, genuinely useful extension. Over time, this benign extension builds a strong reputation, amasses positive user reviews, and acquires a large, trusting user base.

Once a sufficient installation footprint is achieved, the adversary executes the attack by pushing a malicious update. This can occur either because the original developer was malicious from the start (employing delayed activation), the developer's credentials were compromised, or the developer sold the rights of the popular extension to a third-party threat actor[2]. The threat model explicitly relies on the browser's silent auto-update mechanism, which seamlessly delivers the malicious payload to millions of users without requiring further interaction. Consequently, the detection challenge shifts from analyzing absolute files to identifying malicious code injected specifically during the update phase. The defender must differentiate between legitimate feature additions, bug fixes, and malicious privilege escalations[2].

Silent Data Exfiltration and Privacy Discrepancies

A separate lineage of research models the threat around the unauthorized, silent exfiltration of highly sensitive user and corporate data. The severity of this threat model was brought to public attention by the *DataSpii* incident of 2019, an independent research report that documented how eight seemingly innocuous browser extensions silently harvested browsing activity, personally identifiable information (PII), and internal corporate local area network (LAN) data from Fortune 500 companies[11]. In the *DataSpii* model, the attacker utilizes dilatory tactics—intentionally delaying malicious execution until specific high-value conditions are met—to evade dynamic analysis sandboxes.

Academic formalizations of this data exfiltration threat are extensive. The IEEE S&P 2023 paper *ExtPrivA* (Automated Auditing of Privacy Policy Compliance in AI Agents and Extensions) models the threat as a fundamental inconsistency between an extension's legally binding, natural language privacy policy and its actual runtime data collection behaviors[12]. The adversary is modeled as a developer who deceives users by claiming minimal data collection in the store dashboard while technically harvesting extensive telemetry. Similarly, *Arcanum* (USENIX Security 2024) models an active exfiltration threat where extensions abuse their DOM and API access to silently route sensitive data to untrusted third-party network sinks[14]. In these models, the adversary's goal is espionage or the illicit monetization of user data, and the defender's goal is to prove that the data leaving the browser was not authorized by the user.

The Benign-but-Buggy (Vulnerable) Model

Not all threats arise from intentional malice on the part of the extension developer. The ACM CCS 2021 paper *DoubleX: Statically Detecting Vulnerable Data Flows in Browser Extensions at Scale* formalizes a scenario where well-intentioned developers inadvertently introduce vulnerabilities due to a misunderstanding of the browser's complex security boundaries[9].

In this threat model, the extension itself is entirely benign, but its internal architecture—specifically its message-passing mechanisms—is flawed. The adversary is an external entity, such as a malicious website that the user visits. Because the vulnerable extension improperly sanitizes inputs received from the web page before passing them to a highly privileged background script, the external attacker can inject payloads that are executed with the extension's elevated system privileges. This can lead to Universal Cross-Site Scripting (UXSS), arbitrary code execution, or the exposure of sensitive browser data like cookies and bookmarks[9]. Here, the detection system is not hunting for malware, but rather for dangerous topological data flows that cross privilege boundaries without sanitization.

Emerging Threats: GenAI Impersonation and Concept Drift

Recent research highlights the rapid exploitation of emerging technology trends, specifically Generative AI. A comprehensive 2025 study on Malicious GenAI Chrome Extensions models attackers who deploy fraudulent AI assistants[18]. These malicious extensions impersonate legitimate, highly popular GenAI tools (such as ChatGPT or DeepSeek) to exploit user trust. Because AI summarization tools inherently require sweeping permissions to read and modify all data on all websites, users readily grant these excessive privileges[19]. The adversary then utilizes these permissions to execute Adversary-in-the-Browser (AiTB) attacks, passive data exfiltration, and unauthorized traffic redirection.

Furthermore, contemporary threat models heavily emphasize "concept drift," a phenomenon where malware authors continuously mutate their code, tactics, and API usage to evade historical machine learning classifiers. As demonstrated in recent literature, static signature-based detection systems and commercial antivirus aggregators like VirusTotal perform exceptionally poorly against novel JavaScript-based extension malware due to this rapid drift, necessitating models that evaluate behavioral intent rather than static byte signatures[20].

Feature Engineering: Static Analysis Methodologies

The extraction of discriminative features is the absolute cornerstone of any machine learning-based malware detection system. Top-tier papers invariably utilize a sophisticated hybrid approach, beginning with static analysis to evaluate the extension without executing its code. Given that extensions are essentially ZIP-compressed directories containing text-based files, static analysis is computationally efficient and highly scalable across hundreds of thousands of samples.

Manifest File Analytics and Permission Shifts

Every browser extension is governed by a manifest.json file, which acts as the architectural blueprint. Static analysis always begins by extracting the requested permissions (e.g., <all_urls>, tabs, cookies, webRequest), the definitions of background scripts, and the Content Security Policy (CSP). In the context of the bait-and-switch threat model, papers like *You've Changed* do not merely look at the absolute permissions; they analyze the *shifts* in permissions between version updates[2]. An update that suddenly requests the cookies permission when the previous version only required activeTab is a high-fidelity static indicator of potential malicious intent. Furthermore, analyzing the manifest allows researchers to identify the enforcement of Manifest V3, tracking how attackers adapt to the deprecation of blocking web requests and the enforcement of declarative network routing[6].

Abstract Syntax Tree (AST) Deltas

Analyzing raw JavaScript source code is highly prone to error due to code minification, obfuscation, and stylistic formatting differences. To rigorously detect malicious updates, the authors of *You've Changed* parse the JavaScript source code of both the benign and updated versions into Abstract Syntax Trees (ASTs)[2]. An AST strips away syntactic sugar and whitespace, representing the raw logical structure of the program.

By computing the structural differences (the delta or diff) between the two ASTs, the system isolates the exact mathematical sub-graphs representing the newly injected code. This allows the machine learning model to focus purely on the behavioral logic added during the update, ignoring superficial changes caused by different minifiers or bundlers (like Webpack or Rollup). If the AST diff reveals the sudden introduction of dynamic code evaluation nodes (e.g., eval()) or complex string decoding loops, it heavily weights the feature vector toward a malicious classification.

Extension Dependence Graphs (EDG)

To combat the threat of vulnerable data flows, the researchers behind *DoubleX* pioneered the creation of the Extension Dependence Graph (EDG)[9]. Standard static analyzers fail when applied to browser extensions because the execution environment is highly fragmented; a content script and a background page operate in completely isolated execution contexts. A standard control flow graph cannot trace a variable as it moves from one context to another. The EDG overcomes this by providing a comprehensive semantic abstraction of the extension code. It integrates standard control flows and data flows with advanced pointer analysis, and critically, it mathematically models the message-passing interactions (e.g., chrome.runtime.sendMessage and window.postMessage) both within the extension and between the extension and external web pages[5]. By traversing this graph, *DoubleX* can

statically prove the existence of an uninterrupted path between an attacker-controllable input sink and a highly sensitive browser API, extracting this topological path as a primary feature.

Natural Language Processing (NLP) for Policy Alignment

Evaluating privacy compliance requires comparing executable code against human-readable text. *ExtPrivA* and the related *AudAgent* framework rely heavily on Natural Language Processing (NLP) to parse legally binding privacy policies and the standardized developer disclosures found on the Chrome Web Store dashboard[12]. Using sophisticated NLP embeddings and cross-LLM voting mechanisms, these systems extract formal, machine-checkable policy models that categorize exactly what data the developer claims to collect, process, and retain[23].

Similarly, the *CoTH* framework applies NLP to the textual feedback provided by users in CWS reviews. By analyzing sentiment and identifying specific keywords related to hijacking or unwanted advertisements, the system detects sudden spikes in negative anomalies—a strong lagging indicator that a recently pushed update contained a malicious payload[1].

Feature Engineering: Dynamic Analysis Methodologies

While static analysis is scalable, JavaScript is an inherently dynamic language. Malicious logic is frequently obfuscated, constructed at runtime, or fetched from remote Command and Control (C2) servers. Therefore, dynamic analysis—executing the extension within an instrumented, sandboxed browser environment—is necessary to capture the true behavioral footprint of the malware.

HoneyPages and Eliciting Dormant Behavior

A primary challenge in dynamic analysis is that malicious extensions often employ dilatory tactics, remaining dormant to avoid detection during automated sandbox runs. They may wait for a specific amount of time to elapse, or they may only activate when the user visits a high-value target, such as a banking portal or a cryptocurrency exchange.

To overcome this, the USENIX Security 2014 paper *Hulk* pioneered the use of HoneyPages[25]. A HoneyPage is a dynamically generated web page that actively adapts its structure and content to match the expectations of the extension. By interposing on the extension's DOM queries, *Hulk* automatically synthesizes the exact HTML elements and DOM tree structures the extension is searching for. Combined with event handler fuzzing—which systematically forces the execution of all registered event handlers across millions of simulated URLs—this methodology tricks the dormant malware into believing it is operating in a victim's environment, thereby eliciting the malicious payload for observation[25].

Dynamic Taint Tracking and Contextual Flows

To definitively prove data exfiltration, recent frameworks employ dynamic taint tracking. As detailed in *Arcanum* (USENIX 2024) and *CodeX* (CODASPY 2025), taint tracking involves marking highly sensitive data sources within the browser—such as document.cookie, password input fields, browsing history, and bookmark APIs—with a digital "taint"[14].

The modified JavaScript engine then tracks the propagation of these tainted variables as they are processed by the extension, assigned to new variables, or passed through various functions. The system monitors critical exit points, known as network sinks, such as the fetch()

API or XMLHttpRequest. If a tainted variable reaches a network sink without proper authorization, the dynamic analysis engine records a definitive data exfiltration event[15]. This provides highly contextual features, identifying exactly what type of data was stolen and the specific URL of the attacker's endpoint.

Network Topology and Evasion Detection

Dynamic sandboxes rigorously record all outbound network requests initiated by the extension. Feature extraction algorithms analyze these network logs to identify communications with Command and Control (C2) servers. Features are generated based on the reputation of the destination URLs, flagging communications with Newly Registered Domains (NRDs), known malicious IP addresses, or domains generated by Domain Generation Algorithms (DGAs)[22]. Furthermore, dynamic analysis logs errors indicating evasion attempts, such as an extension repeatedly checking for the existence of virtual machine artifacts or specific debugger environments, which strongly correlates with malicious intent[22].

Dataset Curation, Labeling, and Ground Truth

The validity and generalizability of any machine learning model are strictly bound to the quality of its training dataset. Constructing datasets for browser extension security is notoriously difficult due to the ephemeral nature of malware; malicious extensions are frequently removed from the store, making historical analysis challenging.

Scale and Sourcing of Academic Datasets

Top-tier papers overcome this by developing custom, continuous crawling infrastructure to archive the Chrome Web Store over long periods.

- To analyze update deltas effectively, the authors of *You've Changed* compiled an unprecedented dataset of over 156,000 extensions, archiving multiple historical versions of each to track precise changes over time[1].
- For their static vulnerability analysis, *DoubleX* analyzed a massive corpus of 154,484 Chrome extensions to detect vulnerable data flows at an ecosystem scale[9].
- *Arcanum* deployed its dynamic taint tracking system across an archived dataset of over 100,000 functional extensions, evaluating them against popular websites[15].
- Studies focusing on specific emerging threats rely on highly targeted curation. For example, the 2025 GenAI extension study curated a precise dataset of 5,551 AI-themed extensions released over a specific nine-month window to measure the immediate impact of the generative AI boom[19].

The Challenge of Ground Truth and Labeling

In supervised machine learning, accurately labeling samples as either benign or malicious is the most critical step. Top-tier researchers explicitly acknowledge that relying solely on commercial antivirus aggregators, such as VirusTotal, is fundamentally flawed for browser extensions. VirusTotal and similar engines are heavily optimized for compiled binaries (PE, ELF) and mobile application packages (APK). They frequently fail to analyze the highly dynamic, distributed nature of JavaScript extensions, leading to massive false-negative rates and inaccurate ground truth[20].

To establish rigorous, defensible ground truth, academic researchers utilize sophisticated hybrid labeling strategies:

1. **Google Vetting and Takedowns:** Researchers use the Chrome Web Store's own enforcement actions as a lagging indicator of maliciousness. Extensions that are forcibly removed or delisted by Google are flagged, though researchers carefully filter out extensions removed for benign reasons (like developer unpublishing)[1].
2. **Third-Party Threat Intelligence:** Datasets are augmented by cross-referencing extension IDs with reports from dedicated threat intelligence firms (e.g., Duo Security, Avast, DomainTools) and historical repositories like Chrome-Stats[18].
3. **Manual Human Verification:** To guarantee the integrity of their findings, papers like *Arcanum*, *DoubleX*, and the GenAI study dedicate significant manual labor to reviewing flagged extensions. Human analysts manually execute the extensions, review the dynamically injected code, and verify the malicious nature of the network traffic to definitively confirm false positive rates[9].

The CIC Paradigm and Class Imbalance

In the wild, malicious extensions represent a tiny fraction of the overall ecosystem, leading to severe class imbalance. If a model is trained on raw, in-the-wild distributions (e.g., 99% benign, 1% malicious), it will likely achieve 99% accuracy simply by classifying every extension as benign, rendering it useless.

The Canadian Institute for Cybersecurity (CIC) at the University of New Brunswick is renowned for establishing rigorous methodologies to address this in their datasets, such as *CICMalDroid 2020*[32]. While *CICMalDroid* focuses on Android applications, the methodology translates directly to browser extensions. The CIC methodology emphasizes extreme class balance in the training sets (e.g., balancing Adware, Banking Trojans, SMS malware, and Benign apps equally), rigorous static feature extraction (using tools like AndroPyTool), and critical temporal separation[32]. For instance, capturing feature states both before and after a virtual machine reboot counters delayed activation evasion tactics[32]. Extension security researchers adopt these paradigms, utilizing techniques like the Synthetic Minority Over-sampling Technique (SMOTE) to artificially balance their training sets, ensuring the model learns the boundary conditions rather than just the majority class[34].

Machine Learning Models and Theoretical Justifications

Security research favors specific families of machine learning algorithms. While Deep Learning and complex neural networks dominate fields like computer vision, the tabular, high-dimensional, and often sparse data extracted from extension manifests and ASTs is generally best served by robust ensemble methods and specialized embedding techniques.

Ensemble Decision Trees (Random Forest and XGBoost)

Papers in this domain consistently select Random Forest (RF) and eXtreme Gradient Boosting (XGBoost) as their primary detection classifiers[3]. The academic justification for this selection is deeply rooted in the nature of the extracted features:

1. **Heterogeneous Data Handling:** Extension feature vectors are highly heterogeneous. They contain a mix of boolean values (e.g., has_cookies_permission_changed), integer counts (e.g., number_of_eval_calls), and continuous statistical variables (e.g., AST_diff_complexity_score). Tree-based ensemble models natively handle unnormalized, mixed-type data without requiring the extensive, sensitive preprocessing and scaling required by Neural Networks[35].
2. **Interpretability and Explainability:** In the domain of cybersecurity, "black box" models are often unacceptable. Security analysts require explainability; they must understand exactly *why* a model flagged an extension as a threat. Random Forest and XGBoost provide inherent feature importance metrics (such as Gini impurity reduction), allowing researchers to trace a detection back to a specific API call, a dangerous permission shift, or a specific structural change in the AST[35].
3. **Resilience to Sparsity and High Dimensionality:** Features extracted from the presence of thousands of potential API calls and vast AST node variations result in highly sparse, high-dimensional matrices. Random Forest mitigates the risk of overfitting in these environments through bagging (bootstrap aggregating) and random feature subset selection at each node split[35].

Vector Embeddings and Zero-Shot Learning (ZSL)

To combat the rapid evolution of malware—the aforementioned "concept drift"—advanced papers employ vector embeddings combined with Zero-Shot Learning (ZSL). *You've Changed* justifies the use of ZSL because malware authors constantly invent new classes of malicious behavior[1]. Traditional supervised models require complete retraining when encountering a novel malware class. ZSL, however, maps both known and unseen extensions into a continuous semantic vector space. When a new extension is published, the model calculates its distance to known clusters of malicious behavior. If the new extension behaves similarly to a known threat—even if its exact code signature is novel—it is flagged, eliminating the need for constant, manual dataset retraining[1].

NLP and Large Language Models

For features derived from text, highly specialized models are required. The *CoTH* framework utilizes NLP embeddings to process user reviews, while *ExtPrivA* employs models to comprehend the semantics of privacy policies[1]. More recently, frameworks like *AudAgent* have integrated Large Language Models (LLMs) to formalize natural language privacy policies into strict, machine-checkable constraints, utilizing cross-LLM voting mechanisms to reconcile policy elements and ensure high-confidence parsing, which is then compared against runtime data exfiltration annotations[23].

Evaluation Protocols and Experimental Rigor

The hallmark of a top-tier security paper from venues like IEEE S&P or USENIX Security is not merely the proposed architecture, but the rigorous, highly skeptical evaluation protocol used to validate it.

Metrics Beyond Accuracy: Top researchers heavily discourage the use of overall accuracy as a primary metric due to the severe class imbalance inherent in real-world security

deployments. Instead, evaluations rely strictly on Precision, Recall, F1-Score, and False Positive Rates (FPR). For a system to be practically deployable by a store moderator, it must have a high precision. For instance, *ExtPrivA* achieved an 85% precision in detecting inconsistencies[12], while *DoubleX* empirically verified that 89% of its flagged data flows were genuinely exploitable by an external attacker[9]. High precision ensures that human security analysts are not overwhelmed by alert fatigue.

Temporal Cross-Validation against Concept Drift: Standard K-fold cross-validation involves randomly splitting a dataset into training and testing sets. However, in security, this leads to temporal data leakage, as the model may train on malware from 2023 to predict malware from 2020. Because of concept drift[20], top papers employ strict temporal splits: they train their models exclusively on extensions published between, for example, 2017 and 2020, and test them exclusively on unseen extensions from 2021 to 2023. This rigorous protocol proves the model's ability to generalize and detect *future* malware variants, rather than simply memorizing historical data.

Ablation Studies: To scientifically prove that every component of a complex, multi-stage architecture is necessary, researchers conduct ablation studies. They systematically disable individual feature sets—for example, removing the NLP policy analysis, or disabling the dynamic network taint tracking—and measure the resulting degradation in the F1-score. This isolates and quantifies the exact predictive contribution of the AST diffs or the Extension Dependence Graph.

Commercial Baselines: Finally, novel academic systems are benchmarked against industry standards. Papers frequently test their datasets against commercial engines (like VirusTotal) to explicitly demonstrate that existing, commercially available solutions fail where the proposed academic machine learning model succeeds, thereby justifying the novelty and necessity of the research[20].

Comparative Analysis of Key Security Papers

The following table synthesizes the core methodologies of the key papers discussed, providing a structured, comparative blueprint of the experimental designs used in top-tier browser extension security research.

Paper / Venue	Threat Model Focus	Key Features (Static + Dynamic)	Dataset & Labeling	ML Models & Techniques	Evaluation Protocol
You've Changed *(ACM CCS 2020)* [cite: 1, 2]	Bait-and-switch updates; benign extensions turning malicious	**Static:** AST diffs between versions, manifest permission shifts, NLP	156,000+ CWS extensions. Labeled via Google takedowns and AV	RF, SVM, XGBoost, Zero-Shot Learning (ZSL) embeddings, NLP	10-fold cross-validation. Metrics: Accuracy (96.5%), FP Rate (2-3%).

	over time to bypass store vetting.	on user reviews. **Dynamic:** API hooking.	engines.	(CoTH).	Focus on clustering similar updates.
Hulk *(USENIX 2014)* [cite: 25, 26]	Highly evasive malware requiring specific DOM triggers (e.g., banking sites) to activate.	**Dynamic only:** HoneyPages (DOM adaptation), event handler fuzzing, execution traces.	48,000 Chrome extensions. Evaluated by driving 1M URLs.	N/A (Rule-based / Heuristic dynamic analysis).	Dynamic behavioral elicitation. Identified botnets affecting 5.5M users. No traditional ML.
ExtPrivA *(IEEE S&P 2023)* [cite: 12]	Inconsistencies between stated privacy policies and actual runtime data exfiltration.	**Static:** NLP extraction of privacy policies. **Dynamic:** Network request interception, user interaction emulation.	47,200 extensions. Human verification for ground truth.	NLP for policy parsing, deterministic network heuristics.	End-to-end evaluation. 85% precision. Flagged 820 extensions with 1,290 inconsistent flows.
DoubleX *(ACM CCS 2021)* [cite: 9, 16]	Vulnerable extensions enabling external attackers to inject arbitrary data flows.	**Static only:** Extension Dependence Graph (EDG), control/data flow, pointer analysis across	154,484 extensions. Ground truth from known vulnerable sets.	Graph traversal, advanced semantic taint analysis.	Recall on ground truth (92.6%). Precision verification (89% exploitable). Performance profiling

		boundaries.			(med: 2.5s).
Arcanum *(USENIX 2024)* [cite: 14, 15]	Privacy risks; unauthorized DOM/API data flows silently exfiltrated to network sinks.	**Dynamic:** Advanced taint tracking from sensitive DOM/APIs to network boundaries.	100,000+ extensions evaluated across 7 highly popular websites.	Taint flow tracking heuristics.	Large-scale in-the-wild measurement. Flagged >3,000 extensions violating privacy norms.
DataSpii *(2019 / Jadali)* [cite: 11]	Silent, massive corporate data exfiltration using dilatory tactics and evasive techniques.	**Dynamic:** Honeypot traffic analysis, LAN boundary monitoring, URL exfiltration tracking.	8 highly popular extensions impacting millions of active users.	Behavioral threat intelligence, manual analysis.	Real-world impact assessment on Fortune 500 networks. Responsible disclosure via media.
Malicious GenAI Ext. *(2025)* [cite: 18, 19, 29]	AiTB attacks, traffic redirection, and AI impersonation exploiting high permissions.	**Static:** MV3 manifest analysis, obfuscated code detection. **Dynamic:** C2 communication, domain reputation.	5,551 AI-themed extensions curated over nine months.	Multi-signal heuristics, threat intelligence correlation.	Identified 154 0-day malicious extensions. Extensive manual validation required.
CICMalDroid 2020 *(UNB CIC)*	(Android Mobile context applicable	**Static:** Permissions, intents, services.	17,341 apps (strictly balanced benign/mal	RF, XGBoost, CNNs on feature	High emphasis on dataset balance,

[cite: 32, 33, 35]	to extensions) Obfuscated, evasive malware.	**Dynamic:** API calls, distinct temporal states.	ware ratios).	images, BERT text embeddings.	robust F1-scores, and temporal evaluation (pre/post-reboot).
CREAM *(2024)* [cite: 39]	Malicious extensions abusing elevated privileges to steal and abuse authentication cookies.	**Dynamic:** Evaluating browser mechanisms to tie instances to cookies via cryptography.	500 confirmed benign / 500 confirmed malicious extensions.	Cryptographic verification protocols.	Efficacy against cookie theft while measuring run-time overheads on legitimate sites.

Checklist: Architectural Techniques Student Projects Often Miss

For an academic project developing an extension update detector, transitioning from a basic, proof-of-concept implementation to a robust, publication-quality tool requires addressing several systemic complexities. Junior researchers frequently overlook the nuances of the browser environment, leading to high false-positive rates and brittle models. The following checklist details the architectural techniques employed by expert researchers to ensure structural integrity and academic rigor.

Common Student Pitfall	The Expert Security Approach	Implementation Recommendation for Academic Projects
Ignoring the Multi-Component Architecture	Analyzing extension JavaScript as a single monolithic block fails because content scripts cannot directly access highly privileged APIs (like	Implement structural graphs similar to the **Extension Dependence Graph (EDG)** seen in *DoubleX*. Your static analyzer must map data

	chrome.cookies); they must pass messages to background service workers.	flows *across* privilege boundaries via chrome.runtime.sendMessage[9]. Failing to track inter-component messages guarantees massive false negatives.
Failing to Isolate the "Update Delta"	Classifying the entirety of an updated extension from scratch introduces noise, as bait-and-switch malware actively hides within massive, otherwise benign codebases.	Follow the methodology of *You've Changed*. Do not just compare text files; compute the **Abstract Syntax Tree (AST) diff** between the trusted version and the updated version[2]. Your model's feature vector should be extracted specifically from the *delta* (the mathematical subgraph of injected logic).
Underestimating Dilatory Evasion Tactics	Running a dynamic analysis sandbox for a fixed 10 seconds and assuming no network traffic equals a benign extension.	Malware authors routinely use setTimeout or wait for specific high-value DOM interactions before launching payloads (as seen in *DataSpii*)[11]. Implement **HoneyPages** and **Event Handler Fuzzing** (pioneered by *Hulk*) to spoof user interactions and trick dormant malware into executing[25].
Ignoring Manifest V3 Constraints	Building a threat model around deprecated Manifest V2 capabilities, such as expecting extensions to easily execute remotely hosted code or use synchronous	Acknowledge the current ecosystem constraints. A modern detector must specifically parse the manifest for **Manifest V3 evasion techniques**, such as attackers attempting to exploit relaxed Content

	webRequest blocking.	Security Policies (CSP) or abusing the newer declarativeNetRequest ruleset allocations[6].
Temporal Data Leakage in Evaluation	Randomly shuffling a dataset and performing a standard 80/20 train/test split, which allows the model to train on future malware to predict past malware.	Because malware evolves rapidly (concept drift), you must employ strict **Temporal Splits**[20]. Train your model on historical data (e.g., Year N) and test it exclusively on future data (e.g., Year $N+$). This scientifically validates the model's predictive utility in the real world.
Lack of Ablation Studies	Presenting a final F1-score for a complex, multi-stage architecture and concluding the paper without justifying the component complexity.	If your architecture combines static AST diffs with dynamic taint tracking, you must perform an **Ablation Study**. Systematically disable individual feature sets and measure the F1-score degradation to mathematically prove that every component of your architecture is strictly necessary.

Conclusion

The detection of malicious browser extensions is a continuous, escalating arms race characterized by a decisive shift from simple permission-abuse heuristics to highly complex, hybrid program analysis and machine learning methodologies. As evidenced by the exhaustive literature from top-tier venues like IEEE S&P, USENIX Security, and ACM CCS, the most effective systems absolutely refuse to rely on static byte signatures. Instead, they rigorously model the specific, operational realities of the browser ecosystem.

Whether an architecture tracks the mathematical AST deltas between updates[2], executes semantic taint analysis across isolated browser execution contexts[9], or aligns runtime data exfiltration with natural language privacy policies[12], success requires deep domain expertise.

For an academic project targeting malware-update detection, achieving publication-quality results relies on formally adopting the "bait-and-switch" threat model, implementing AST-aware delta feature extraction, simulating user interactions to bypass dilatory tactics, and critically evaluating the machine learning pipeline against the inevitable reality of concept drift using temporal splits. By adhering to the architectural blueprints and rigorous evaluation protocols detailed in this report, researchers can engineer resilient systems capable of systematically securing the modern web environment against highly sophisticated adversaries.

Works cited

1. Detecting Malicious Browser Extensions through their Update Deltas, https://www.researchgate.net/publication/349829581_You've_Changed_Detecting_Malicious_Browser_Extensions_through_their_Update_Deltas
2. Detecting Malicious Browser Extensions through their Update Deltas, http://www.mathcs.richmond.edu/~dszajda/classes/cs334/Spring_2021/papers/pantelaios_Detecting_Malicious_Browser_Extensions.pdf
3. Detecting Malicious Browser Extensions by Combining Machine, https://www.researchgate.net/publication/370587512_Detecting_Malicious_Browser_Extensions_by_Combining_Machine_Learning_and_Feature_Engineering
4. Did I Vet You Before? Assessing the Chrome Web Store ... - arXiv, https://arxiv.org/html/2406.00374v2
5. Statically Detecting Vulnerable Data Flows in Browser Extensions at, https://www.researchgate.net/publication/356200946_DoubleX_Statically_Detecting_Vulnerable_Data_Flows_in_Browser_Extensions_at_Scale
6. Manifest V3 Unveiled: Navigating the New Era of Browser Extensions, https://arxiv.org/html/2404.08310v1
7. Manifest V3 Unveiled: Navigating the New Era of Browser Extensions, https://arxiv.org/pdf/2404.08310
8. COMBAT: Continuous Monitoring of Browser Extensions against, https://www.atlantis-press.com/article/126024630.pdf
9. DoubleX: Statically Detecting Vulnerable Data Flows in Browser, https://swag.cispa.saarland/papers/fass2021doublex.pdf
10. The TROJANIZED COMPONENT - AliBawazeEer - Medium, https://abawazeeer.medium.com/the-trojanized-component-dissecting-the-browser-extension-and-cti-great-effects-8cca18cd1a31
11. DataSpii - A global catastrophic data leak via browser extensions, https://securitywithsam.com/2019/07/dataspii-leak-via-browser-extensions/
12. Five papers by CSE researchers presented at IEEE S&P conference, https://cse.engin.umich.edu/stories/five-papers-by-cse-researchers-presented-at-ieee-sp-conference
13. MiniScope: Automated UI Exploration and Privacy Inconsistency, https://arxiv.org/html/2401.03218v3
14. Arcanum: Detecting and Evaluating the Privacy Risks of Browser, https://www.usenix.org/system/files/usenixsecurity24-xie-qinge.pdf
15. USENIX Security 2024 – Research Impact & Leadership,

https://sites.gatech.edu/research/usenix-security-2024/
16. Statically Detecting Vulnerable Data Flows in Browser Extensions at, https://www.semanticscholar.org/paper/DoubleX%3A-Statically-Detecting-Vulnerable-Data-Flows-Fass-Som%C3%A9/9faf3ae01419b62d7269ce95f0579d6e2b3bc55c
17. Hardening the security analysis of browser extensions, https://research.chalmers.se/publication/530531/file/530531_Fulltext.pdf
18. Malicious GenAI Chrome Extensions: Unpacking Data Exfiltration, https://www.alphaxiv.org/abs/2512.10029
19. UNPACKING DATA EXFILTRATION AND MALICIOUS BEHAVIOURS, https://arxiv.org/pdf/2512.10029
20. Peeking through the window: Fingerprinting Browser Extensions, https://www.researchgate.net/publication/386586698_Peeking_through_the_window_Fingerprinting_Browser_Extensions_through_Page-Visible_Execution_Traces_and_Interactions
21. It's not Easy: Applying Supervised Machine Learning to Detect, https://arxiv.org/abs/2509.21590
22. FV8: A Forced Execution JavaScript Engine for Detecting Evasive, https://arxiv.org/html/2405.13175v1
23. Automated Auditing of Privacy Policy Compliance in AI Agents, https://www.researchgate.net/publication/397521574_AudAgent_Automated_Auditing_of_Privacy_Policy_Compliance_in_AI_Agents
24. AudAgent: Automated Auditing of Privacy Policy Compliance in AI, https://arxiv.org/html/2511.07441
25. Hulk: Eliciting Malicious Behavior in Browser Extensions - ICIR, https://www.icir.org/vern/papers/hulk-usesec14.pdf
26. Hulk: Eliciting Malicious Behavior in Browser Extensions, https://www.researchgate.net/publication/264991585_Hulk_Eliciting_Malicious_Behavior_in_Browser_Extensions
27. CodeX: Contextual Flow Tracking for Browser Extensions, https://www.cse.chalmers.se/~andrei/codaspy25.pdf
28. CodeX: Contextual Flow Tracking for Browser Extensions, https://www.plai.ifi.lmu.de/publications/codaspy25-codex.pdf
29. (PDF) Malicious GenAI Chrome Extensions: Unpacking Data, https://www.researchgate.net/publication/398602720_Malicious_GenAI_Chrome_Extensions_Unpacking_Data_Exfiltration_and_Malicious_Behaviours
30. It's not Easy: Applying Supervised Machine Learning to Detect, https://aurore54f.github.io/papers/2025-tweb_rosenzweig_extensionsconcdrift.pdf
31. 500 Chrome Extensions Caught Stealing Private Data of 1.7 Million, https://thehackernews.com/2020/02/chrome-extension-malware.html
32. Visualising Static Features and Classifying Android Malware Using, https://www.mdpi.com/2076-3417/14/11/4772
33. Modern Mobile Malware Detection Framework Using Machine, https://www.techscience.com/csse/v48n5/57946/html

34. Detecting Malicious Browser Extensions by Combining Machine, https://www.scilit.com/publications/d982eed47f3117bad0e4b2f3bf1ca9d6
35. Detecting Malware by Analyzing App Permissions on Android Platform, https://pmc.ncbi.nlm.nih.gov/articles/PMC9609682/
36. Artificial Intelligence in Cyberspace Security - MDPI, https://mdpi-res.com/bookfiles/book/13135/Artificial_Intelligence_in_Cyberspace_Security.pdf?v=1788854848
37. APT attack detection based on flow network analysis techniques, https://www.researchgate.net/publication/342771321_APT_attack_detection_based_on_flow_network_analysis_techniques_using_deep_learning
38. Android Mobile Malware Detection Using Machine Learning, https://www.researchgate.net/publication/353034728_Android_Mobile_Malware_Detection_Using_Machine_Learning_A_Systematic_Review
39. Towards Browser Controls to Protect Cookies from Malicious ... - arXiv, https://arxiv.org/html/2405.06830v2